Université de Limoges

IDS : Intrusion Detection System

Olivier Blazy & Emmanuel Conchon
(emmanuel.conchon@unilim.fr)

Université de Limoges



Université de Limoges

Petit historique

- Premiers travaux par Anderson (80) et Denning (87)
  - Travaux initiés par le DoD
  - Milieu 80 à milieu 90, début 2000 : purement du domaine de la recherche
- Depuis : commercialisation et toujours (beaucoup de) travaux de recherche
- Fin des années 2000 : convergence avec les domaines de la lutte anti-virus et de l'analyse de malware
- Récemment : intégration dans la **supervision de sécurité**

Université de Limoges

Définitions de la RFC 4949

> **Security intrusion**: A security event, or a combination of multiple security events, that constitutes a security incident in which an intruder gains, or attempts to gain, access to a system or system resource without having authorization to do so.

> **Intrusion detection**: Sensing and analyzing system events for the purpose of noticing (i.e., becoming aware of) attempts to access system resources in an unauthorized manner.

La RFC distingue la détection **active** (temps réel, réponse immédiate) de la détection **passive** (analyse a posteriori des journaux)

Université de Limoges

Démarche SSI

- Phase 1 : **analyse de risques**
  - Identifier les ressources à protéger
  - Identifier les menaces qui pèsent sur ces ressources
  - Évaluer la vraisemblance de ces menaces
- Phase 2 : **définition d'une politique de sécurité**
  - Qui est autorisé à utiliser la ressource et comment ?
  - Qui est autorisé à accorder des droits sur la ressource ?
  - Qui possède les privilèges de l'administrateur ?

Démarche SSI

- Phase 3 : **mise en œuvre de la politique de sécurité**
  - Choix de l'ensemble des mécanismes les plus simples possibles permettant de protéger les ressources de la manière la plus efficace et pour un coût acceptable
  - Approches préventives
  - Surveillance a posteriori : **détection d'intrusions**

> La prévention ne suffit pas : une partie des attaques vient de l'**intérieur** (employés, comptes légitimes compromis), d'où la nécessité de surveiller aussi les hôtes

Université de Limoges

Les différents types d'IDS

Il existe deux principaux types d'IDS :

- **HIDS** (*Host-based IDS*) : surveille les événements d'un hôte
- **NIDS** (*Network-based IDS*) : surveille le trafic réseau et analyse les protocoles

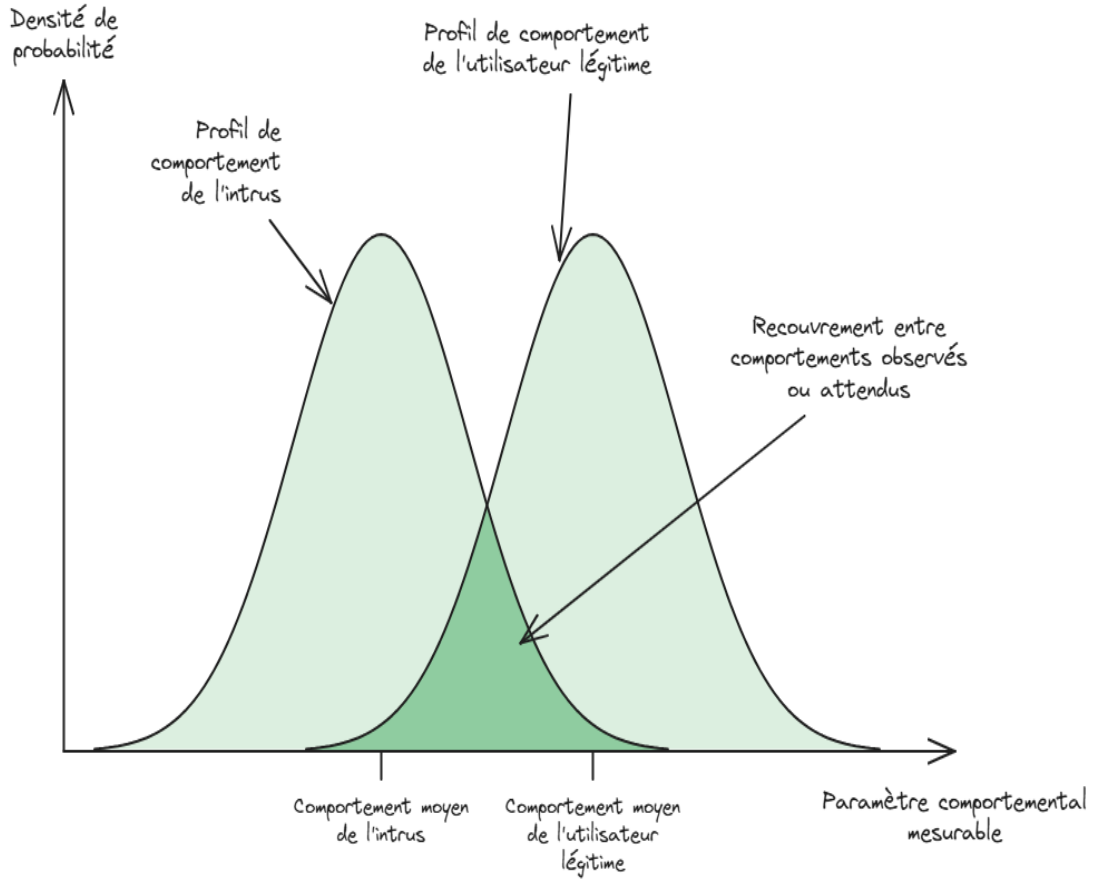
Un **IPS** (*Intrusion Prevention System*) est un IDS placé **en coupure**, capable de bloquer ce qu'il détecte

Dans tous les cas, un IDS est composé de :

- **Capteurs** : collectent les données
- **Analyseurs** : déterminent si une intrusion a eu lieu
- **Interface utilisateur** : permet de visualiser les résultats

Principe de fonctionnement

- Suppose que le comportement d'un intrus diffère de celui d'un utilisateur
  - Les recouvrements posent problème
  - **Faux positifs / négatifs**



Université de Limoges

Pré-requis d'un IDS

- **Fiabilité**
  - Tourne en permanence
  - Résiste aux erreurs et aux tentatives de corruption
- **Performance**
  - Impose un surcoût minimum au système
  - Entrave le moins possible le service
  - Peut assurer un passage à l'échelle
- **Adaptabilité**
  - Adapté à la politique de sécurité
  - S'adapte aux changements dans le système / chez les utilisateurs
  - Permet une reconfiguration dynamique

Université de Limoges

La détection d'anomalies dans les IDS

Détection d'anomalie

- Détection par seuil
  - Compte le nombre d'occurrences d'un événement sur un intervalle de temps
- Sur profil
  - Établit des profils de l'activité normale de chaque utilisateur et détecte des changements

Détection de signature

- Définit un ensemble de règles / schémas d'attaques pouvant être utilisé pour établir qu'un individu est un intrus

Mesures classiques : login / session

Deux modèles : **statistique** (moyenne et écart type) ou **opérationnel** (seuil fixe)

- Habitudes de connexion (statistique)
  - Connexion à des horaires ou depuis des lieux inhabituels
  - Durée de session anormalement différente
- Activité pendant la session (statistique)
  - Volume / destination des données sortantes → fuite de données
  - Consommation de ressources inhabituelle
- Événements suspects (opérationnel)
  - Connexion sur un compte inactif depuis longtemps
  - Nombreuses tentatives de login → brute force du mot de passe
  - Échecs de login depuis certains terminaux

Mesures classiques : programmes et fichiers

- Exécution de programmes
  - Commandes inhabituelles pour l'utilisateur → compte usurpé (statistique)
  - Consommation de ressources anormale → virus / cheval de Troie (statistique)
  - Échecs d'exécution répétés → tentative d'élévation de privilèges (opérationnel)
- Accès aux fichiers
  - Fréquence anormale de lecture / écriture / création / suppression → compte usurpé (statistique)
  - Accès aux logs → tentative d'altération des traces (statistique)
  - Échecs d'accès répétés → recherche de fichiers interdits (opérationnel)

Création de signatures

> Apprendre le comportement **normal** à partir de l'historique

- Analyse des logs passés → modèles d'utilisation classiques
- Génération des règles à partir de ces données
- Comparaison des comportements observés à ces règles
- Aucune connaissance des vulnérabilités requise
- Nécessite une grande base de données



Université de Limoges

Détection de Signature

> Reconnaître les attaques **connues** grâce à des règles

- Règles décrivant des attaques classiques
- Règles visant l'exploitation de faiblesses connues
- Règles définissant des comportements suspects
- Règles spécifiques à chaque machine / système d'exploitation

Network-Based IDS (NIDS)

- Surveille le trafic à des points choisis du réseau
- Examine le trafic paquet par paquet en (quasi) temps réel
- Examine l'activité de la couche réseau, transport, et/ou applicative
- Inclut un grand nombre de capteurs, au moins un serveur pour la gestion du NIDS, et au moins une console d'administration pour l'interface humaine
- Les analyses peuvent être faites au niveau des senseurs, du serveur ou des deux



Honeypot



- Faux système / leurre mis en place pour
  - Attirer un attaquant potentiel loin des systèmes critiques
  - Collecter des informations sur les activités d'un attaquant
  - Encourager un attaquant à rester suffisamment longtemps dans le système pour que les administrateurs puissent réagir
- Rempli d'informations fabriquées auxquelles un utilisateur honnête n'aurait pas accès
  - **Les ressources n'ont aucune valeur**
  - Les communications entrantes sont vraisemblablement des probes ou attaques
  - Les sortantes suggèrent que le système a été compromis

Université de Limoges

Comparaison entre NIDS et HIDS

NIDS

- Avantages des NIDS
  - Couverture de l'ensemble du réseau
  - Support pour la détection d'un nombre d'attaques conséquent
- Inconvénients des NIDS
  - Taux de faux positifs élevé
  - Analyse des données chiffrées impossible
  - Traitement superficiel de l'information (contrainte de performance)
  - La grande variété de systèmes d'exploitation complique le mimétisme de la part des NIDS

Université de Limoges

Comparaison entre NIDS et HIDS

HIDS

- Avantages des HIDS
  - Pas ou peu de faux positifs
  - Pas ou peu d'adaptation à la politique de sécurité en utilisation
- Inconvénients des HIDS
  - Complexité du déploiement

Université de Limoges

Un exemple de NIDS : SNORT



Université de Limoges

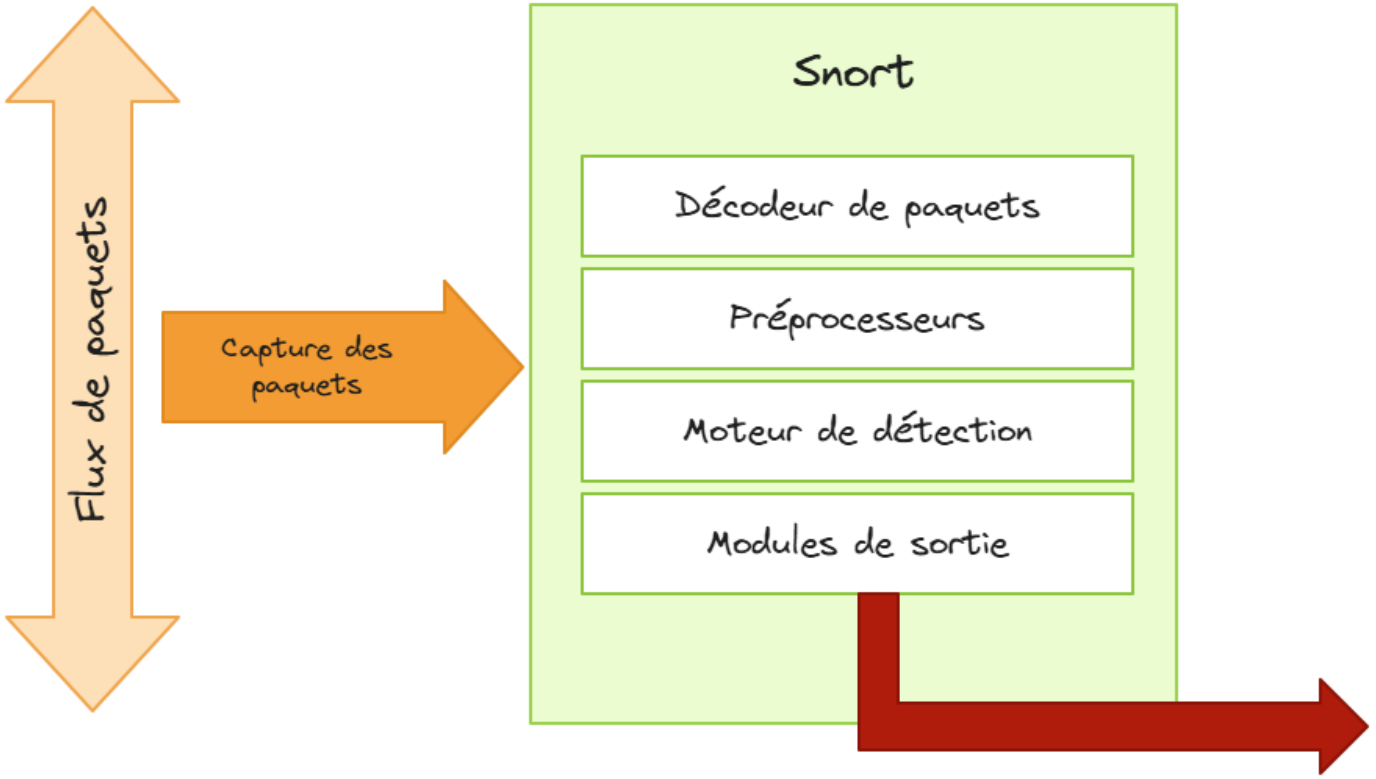
Snort : présentation

> NIDS / IPS open source créé par Martin Roesch en 1998, maintenu par Cisco depuis le rachat de Sourcefire (2013) — https://www.snort.org

- Analyse le trafic en temps réel à l'aide de **règles** (signatures)
- Léger : peu de mémoire et de CPU, facile à déployer et à configurer
- Deux modes de fonctionnement :
  - **passif** (IDS) : analyse une copie du trafic
  - **en coupure** (IPS) : peut bloquer les paquets
- Deux versions : **Snort 2** (présentée ici) et **Snort 3** (réécrit, multi-thread)

Université de Limoges

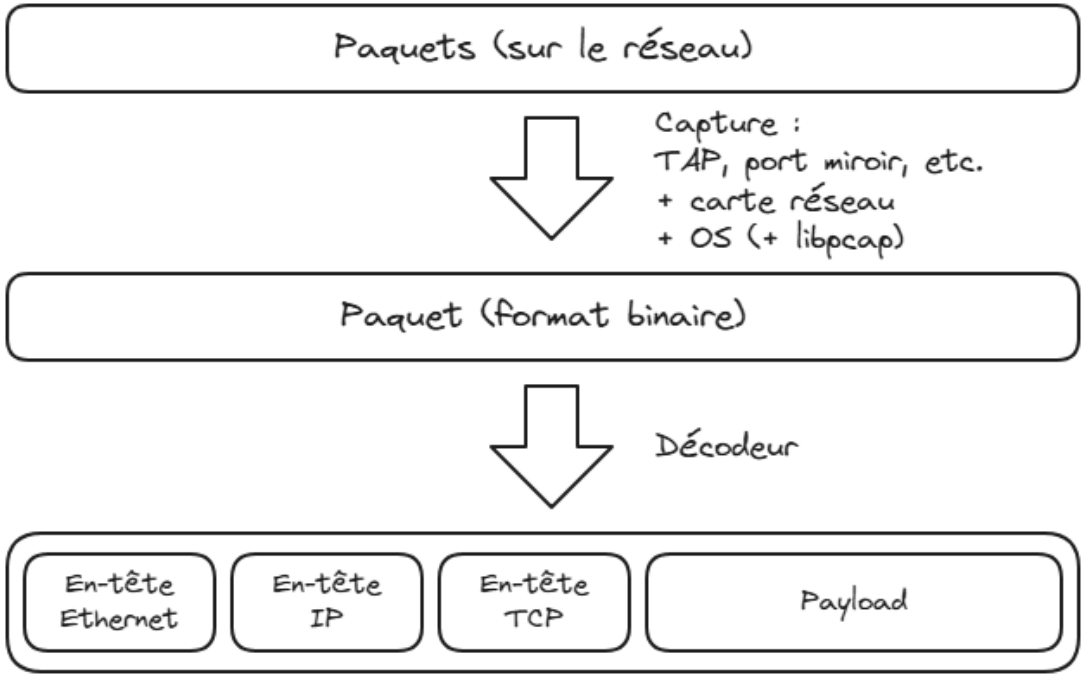
Architecture de Snort (v2)



Chaque paquet capturé traverse quatre étapes : **décodage**, **pré-traitement**, **détection** et **sortie**

Capture et décodage

- Capture via la bibliothèque **DAQ** (libpcap, NFQueue…), en mode passif ou en coupure
- Le **décodeur** analyse les en-têtes (Ethernet, IP, TCP / UDP…)
  - Le paquet est rangé dans une structure interne
  - Vérifications simples : en-têtes malformés, tailles incohérentes → alertes de décodage



Pré-processeurs

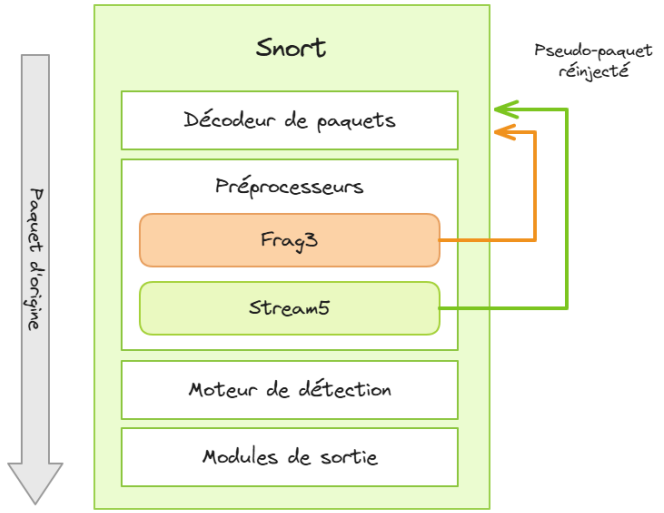
> Sans pré-traitement, un attaquant contourne facilement les signatures

- **Réassemblage** : Frag3 (IP), Stream5 (TCP)
- **Normalisation** : http_inspect (URI, encodages)
- **Anomalies** : scans de ports (sfPortscan)
- Exécutés **dans l'ordre** de la configuration



Frag3 et Stream5

- **Frag3** réassemble les fragments IP
  - Détecte les attaques par fragmentation (chevauchements, fragments minuscules)
  - Imite la politique de ré-assemblage de l'OS cible (Linux, Windows, BSD…)
- **Stream5** réassemble les flux TCP et suit les sessions
  - Détecte les anomalies TCP (segments chevauchants, états incohérents)
  - Configurable par port et par OS cible
- Le résultat est un **pseudo-paquet** réinjecté dans la chaîne



http_inspect

> Une même URL peut s'écrire de multiples façons pour échapper aux signatures

```
/downloads/../cgi-bin/../safe\../downloads/./snort.tar.gz
→ /downloads/snort.tar.gz
```

- Normalise l'URI (`../` , `./` , encodages `%xx` , unicode…) avant la détection
- Les règles inspectent l'URI normalisée (option `http_uri`)
- `flow_depth` : volume de réponse inspecté → compromis faux négatifs / performance

Université de Limoges

Structure d'une règle

```
alert tcp any any -> 192.168.1.0/24 80 (msg:"Accès admin"; content:"/admin"; sid:1000001;)
```

- **En-tête** : ce que l'on surveille
  - Action, protocole (`ip` , `tcp` , `udp` , `icmp`)
  - IP et port source, direction (`->` ou `<>`), IP et port destination
- **Options** (entre parenthèses, séparées par `;`) : ce que l'on cherche
  - Conditions sur le paquet et métadonnées de l'alerte

Université de Limoges

Actions

- `alert` : génère une alerte et journalise le paquet
- `log` : journalise le paquet sans alerte
- `pass` : ignore le paquet
- En mode IPS (en coupure) uniquement :
  - `drop` : bloque et journalise le paquet
  - `reject` : bloque, journalise et renvoie un reset TCP ou une erreur ICMP
  - `sdrop` : bloque sans journaliser

Université de Limoges

Options principales

- **Métadonnées** : `msg` (message), `classtype` (type d'attaque), `sid` / `rev` (identifiant, version)
- **Contenu** : `content` (motif), `nocase`, `offset` / `depth` (zone de recherche), `pcre` (regex)
- **Hors contenu** : `ttl`, `dsize` (taille), `flags` / `seq` (en-tête TCP), `flow` (sens et état de la connexion)
- **Post-détection** : `logto`, `session`, `tag` (journaliser la suite du trafic)

Université de Limoges

Exemple : injection SQL

```
$req = "SELECT * FROM users WHERE name = '" . $user . "';";
```

```
alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS $HTTP_PORTS (msg:"SQL Injection";
      flow:to_server,established; content:".php"; http_uri;
      pcre:"/(\%27)|(\')|(\-\-)|(%23)|(#)/i";
      classtype:web-application-attack; sid:9099; rev:5;)
```

- La regex repère une apostrophe (`'` , `%27`) ou un commentaire SQL (`--` , `#` , `%23`)
- Trop générique : « O'Brien » déclenche une alerte → **faux positif**

Université de Limoges

D'où viennent les signatures ?

- **Règles Talos** (Cisco, ex-VRT de Sourcefire)
  - *Community* : gratuites et libres (GPL)
  - *Registered* : gratuites sur inscription, publiées avec 30 jours de retard
  - *Subscriber* : payantes, disponibles immédiatement
- **Règles tierces** : Emerging Threats (ET Open, ET Pro)
- **Règles locales** écrites par l'administrateur (`local.rules`)

> La qualité de la détection dépend avant tout de celle des signatures

Université de Limoges

Moteur de détection

- Les règles sont organisées en arbre :
  - **RTN** (*Rule Tree Node*) : en-tête commun à plusieurs règles
  - **OTN** (*Option Tree Node*) : options propres à chaque règle
- Pour chaque paquet : sélection des RTN qui correspondent, puis test des OTN
- Recherche de motifs optimisée
  - `content` : recherche multi-motifs (Aho-Corasick)
  - `pcre` : moteur d'expressions régulières, plus coûteux
- Les algorithmes doivent résister aux attaques par complexité algorithmique

Sorties

- Journalisation : syslog, fichiers texte, **unified2** (format binaire rapide)
- Le format unified2 est lu par un outil externe (Barnyard2…) qui alimente une base ou un SIEM
- En mode IPS : blocage effectif des paquets (`drop` , `reject`)
- Extensions dynamiques : pré-processeurs et règles compilées (*so-rules*)

> Les alertes de Snort sont ensuite centralisées et corrélées (cf. Wazuh)

Université de Limoges

Ce qui change avec Snort 3

- **Multi-thread**, configuration en **Lua** (`snort.lua`)
- **Inspectors** au lieu des pré-processeurs, ordonnés par type (`stream_ip` , `stream_tcp` ...)
- **Détection automatique des services** : HTTP reconnu sur n'importe quel port
- **Règles** : *sticky buffers* placés avant le motif (`http_uri; content:".php";`)
- **Actions** : `activate` , `dynamic` et `sdrop` disparaissent ; `block` , `react` et `rewrite` apparaissent
- **Sorties** : `alert_json` , directement exploitable par un SIEM

Université de Limoges

Un exemple d'HIDS : OSSEC



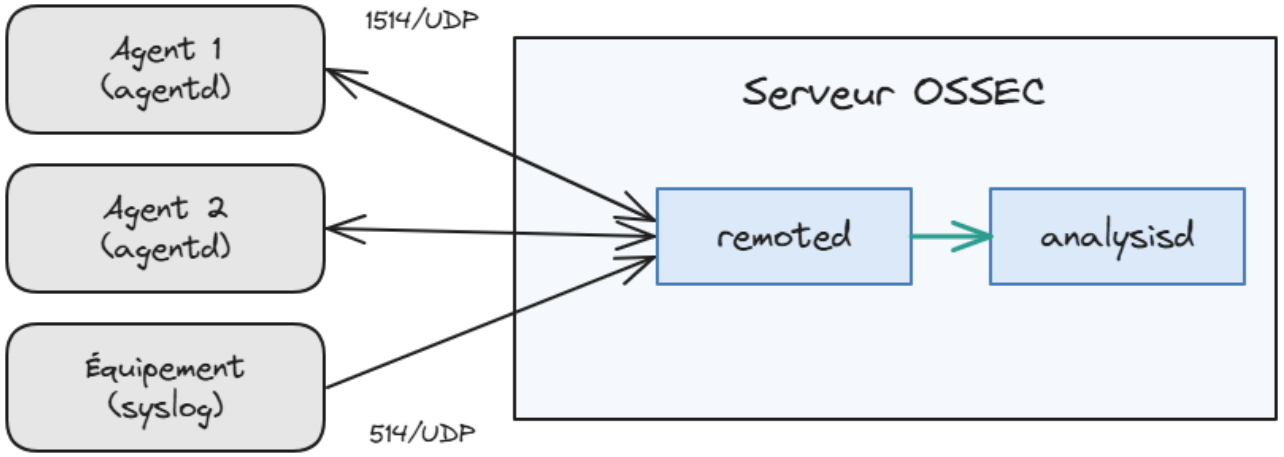
Université de Limoges

OSSEC : Présentation générale

> HIDS **open source** créé par Daniel Cid en 2004, fonctionnant sur Linux, Unix et Windows

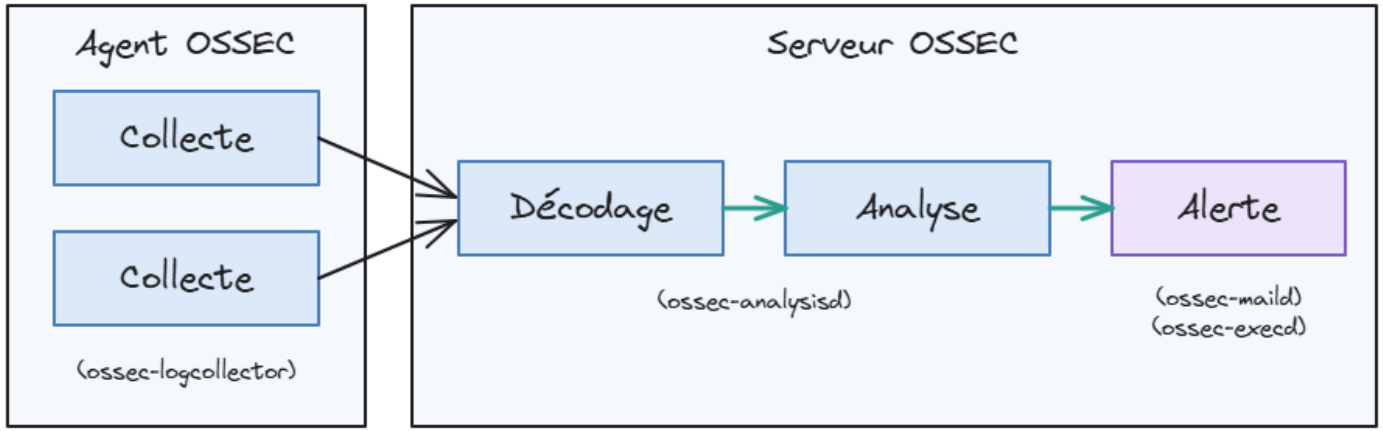
- **Contrôle d'intégrité** des fichiers et du registre (*syscheck*)
- **Analyse des logs** pour détecter les anomalies sur l'hôte
- **Recherche de rootkits** et **audit des politiques** (*rootcheck*)
- **Réponse active** : réaction automatique à une alerte (blocage d'IP…)
- Architecture **agent / serveur** : collecte locale, analyse centralisée
- **Sécurisé par défaut** : installation verrouillée, chaque processus avec le minimum de privilèges

Communication réseau



- Chaque agent (*agentd*) envoie ses événements au démon **remoted** du serveur
- Les équipements sans agent (routeurs, pare-feu) envoient leurs logs en **syslog**
- remoted transmet le tout à **analysisd**, qui centralise l'analyse
- Le lien est bidirectionnel : configuration et réponses actives vers les agents

Communication réseau



- Sur l'agent, **logcollector** lit les logs locaux (fichiers, journaux Windows…)
- Sur le serveur, **analysisd** :
  - **décode** chaque événement (programme, IP source, utilisateur…)
  - l'**analyse** avec les règles, qui lui attribuent un niveau de 0 à 15
- Si une règle se déclenche : **alerte** par mail (*maild*) ou réponse active (*execd*)

Université de Limoges

Recherche de rootkit : principe

> Un rootkit **cache** sa présence : fichiers, processus, ports…

- Il remplace des binaires système (`ls` , `ps` , `netstat`) ou modifie le noyau
- Idée d'OSSEC (module *rootcheck*) : **croiser plusieurs sources**
  - Ce qu'affichent les outils système
  - Ce que révèlent directement les appels système
- Toute **incohérence** entre les deux est suspecte
- Analyse périodique par l'agent, résultats envoyés au serveur

Université de Limoges

Rootcheck : détection par signatures

> Ne détecte que les rootkits **déjà connus**

- Base de **fichiers connus** de rootkits (Adore, Knark, LOC…)
  - Recherchés via plusieurs appels (`stat` , `open` , `opendir`)
  - Vu par un appel mais pas par un autre → rootkit noyau
- Base de **binaires piégés** (chevaux de Troie)
  - Motifs caractéristiques dans `ls` , `ps` , `netstat` , `login` …

Université de Limoges

Rootcheck : détection d'anomalies

- **Système de fichiers**
  - Fichiers ordinaires dans `/dev`, réservé aux périphériques
  - Fichiers SUID, fichiers root modifiables par tous, répertoires cachés
- **Processus cachés** : un PID répond à `kill(pid, 0)` mais est absent de `ps`
- **Ports cachés** : `bind()` échoue sur un port absent de `netstat`
- **Interfaces** : mode *promiscuous* non signalé par `ifconfig` → sniffer

Université de Limoges

Politique de surveillance : principe

> Détecter les **mauvaises configurations** avant qu'elles ne mènent à une brèche

- Audit de configuration (*policy monitoring*), réalisé par `rootcheck`
- L'agent compare l'état de l'hôte à des **politiques** décrites en texte
  - Fournies : benchmarks CIS (https://www.cisecurity.org), audit Unix / Windows
  - Ou écrites par l'administrateur
- Chaque non-conformité génère une alerte envoyée au serveur

Université de Limoges

Politique de surveillance : vérifications

- **Fichiers** (`f:`) : existence, contenu (expressions régulières)
- **Répertoires** (`d:`) : présence de fichiers, contenu
- **Processus** (`p:`) : un service tourne-t-il ?
- **Registre Windows** (`r:`) : présence de clés et de valeurs

Exemples de questions :

- La connexion SSH en root est-elle autorisée ?
- Un anti-virus est-il installé ? En cours d'exécution ?
- Le pare-feu fonctionne-t-il ?

Université de Limoges

Politique de surveillance : exemple

```
[SSH Configuration - 1: Root can log in] [any] [1]
f:/etc/ssh/sshd_config -> !r:^# && r:PermitRootLogin\.+yes;
f:/etc/ssh/sshd_config -> r:^#\s*PermitRootLogin;
```

- En-tête : nom, condition (`any` , `all` , `none`), référence
- Alerte si `PermitRootLogin yes` est actif…
- … ou si la directive est commentée (valeur par défaut appliquée)

> Wazuh a repris ce principe sous le nom **SCA**, avec des politiques en YAML

Règles OSSEC

```
<rule id="2017" level="4">
  <if_sid>2010</if_sid>
  <match>fatal: Timeout before authentication for</match>
  <description>Timeout while logging in (sshd).</description>
</rule>
```

- Écrites en XML, avec un **niveau** de 0 (ignoré) à 15 (critique)
- `if_sid` : testée seulement si la règle parente 2010 a été déclenchée → **arbre de règles**
- `match` : motif recherché dans le log décodé
- Règles **atomiques** (un événement) ou **composites** (plusieurs événements, cf. Wazuh)

Challenges

- Déployer correctement de nombreux agents
- Des attaquants sachant qu'il y a une réponse active peuvent l'exploiter
- Les IP peuvent être spoofées pour déclencher des réponses non adaptées
- Alert flooding → par défaut, OSSEC se limite à 12 alertes par heure
- Injection de logs

> Il faut toujours **retravailler ses règles**

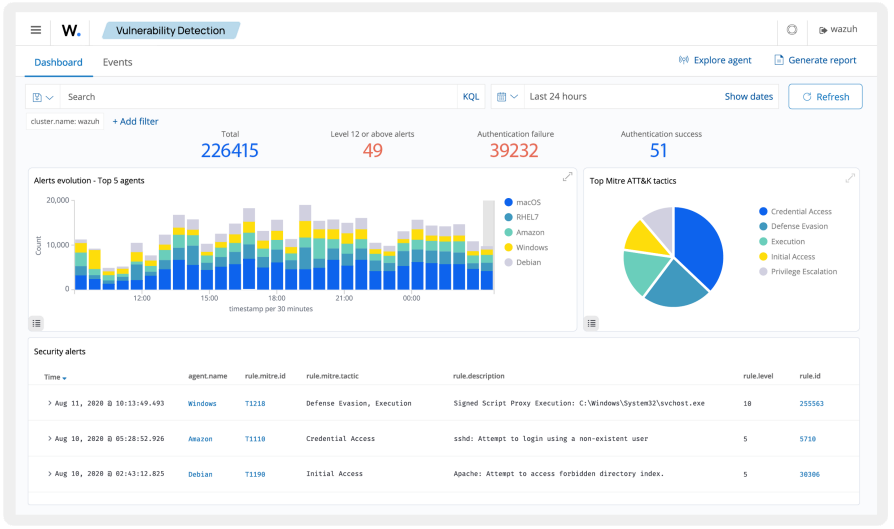
Université de Limoges

IDS hybride : Wazuh

Wazuh

Plateforme open source de détection et de réponse aux menaces

- Fork d'OSSEC créé en 2015, sous licence GPLv2
- Reprend les fonctions HIDS d'OSSEC
  - Intégrité, rootkits, analyse de logs, réponse active
- Ajoute la centralisation, la corrélation et la visualisation
- **Très utilisé dans les SOC comme SIEM**



Université de Limoges

Wazuh : un IDS hybride

> Combiner les avantages des **NIDS** et des **HIDS**

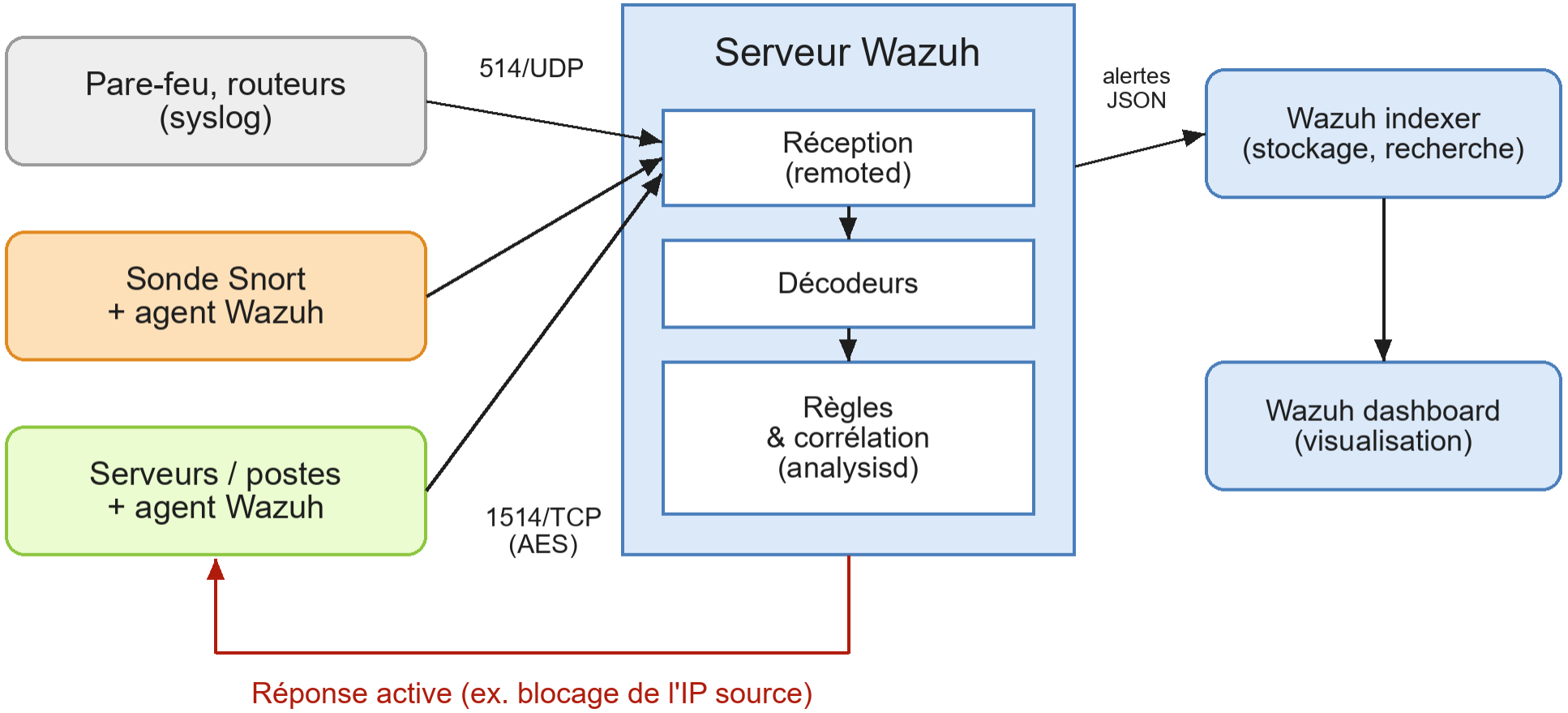
- Sondes réseau (Snort) et agents hôtes placés en des points stratégiques
- Tous les événements remontent vers un serveur central
  - Ils sont décodés dans un format commun (JSON)
  - Un même moteur de règles les analyse et les corrèle
- **On obtient une vision globale de l'état du SI**

Université de Limoges

Wazuh : Principaux composants

- **Agent** : installé sur les hôtes
  - Collecte les logs, contrôle l'intégrité, exécute les réponses actives
- **Serveur** (manager) : cœur de l'analyse
  - Reçoit, décode et applique les règles de détection et de corrélation
- **Indexer** : stockage et recherche des alertes (fork d'OpenSearch)
- **Dashboard** : interface web de visualisation et d'investigation

Architecture avec des sondes Snort



Université de Limoges

Intégrer Snort dans Wazuh

- Snort écrit ses alertes dans un fichier sur la sonde
- L'agent Wazuh de la sonde lit ce fichier (`ossec.conf`)

```
<localfile>
  <log_format>snort-full</log_format>
  <location>/var/log/snort/alert</location>
</localfile>
```

- Le serveur fournit décodeurs et règles pour Snort (groupe `ids`)
- Avec Snort 3 : sortie `alert_json` et `log_format` `json`

Université de Limoges

Corrélation : alertes répétées

```
<rule id="100100" level="12" frequency="8" timeframe="120">
  <if_matched_group>ids</if_matched_group>
  <same_srcip />
  <description>8 alertes Snort de la même IP en 2 min</description>
</rule>
```

- Règle **composite** : déclenchée par une accumulation d'événements
- Une seule alerte de haut niveau au lieu de dizaines d'alertes Snort

Université de Limoges

Corrélation : NIDS + HIDS

```
<rule id="100101" level="14" timeframe="600">
  <if_group>authentication_success</if_group>
  <if_matched_group>ids</if_matched_group>
  <same_srcip />
  <description>Connexion réussie depuis une IP signalée par Snort</description>
</rule>
```

- L'alerte réseau (Snort) est corrélée avec un événement hôte (agent)
- Ni l'une ni l'autre ne suffit seule à révéler l'intrusion

Université de Limoges

Réponse active

- Une règle peut déclencher un script sur un ou plusieurs agents

```
<active-response>
  <command>firewall-drop</command>
  <location>all</location>
  <rules_id>100101</rules_id>
  <timeout>600</timeout>
</active-response>
```

- Ici : blocage de l'IP source sur tous les agents pendant 10 min
- Mêmes risques qu'avec OSSEC : IP spoofées, blocages abusifs

Université de Limoges

Autres fonctionnalités

- Contrôle d'intégrité des fichiers (FIM) en temps réel
- Audit de configuration (SCA) selon les benchmarks CIS
- Détection de vulnérabilités à partir de l'inventaire logiciel
- Correspondance des alertes avec MITRE ATT&CK
- Rapports de conformité : PCI DSS, RGPD, HIPAA…
- Supervision du cloud et des conteneurs

Université de Limoges

Conclusion sur Wazuh

> En centralisant et en corrélant les alertes des sondes réseau et des agents hôtes, Wazuh offre une **vision globale** de la sécurité

- Multiplier les sondes rend le contournement de la détection plus difficile
- Mais le volume d'alertes reste important
  - Il faut **retravailler ses règles** et ses seuils
  - D'où l'importance de la **corrélation d'alertes**

Université de Limoges

Corrélation d'alertes

Université de Limoges

Corrélation : pourquoi ?

Une sonde, aussi bonne soit-elle, ne voit qu'**une partie** de l'attaque.

- **Elle n'observe pas tout** : un NIDS ne voit pas le trafic chiffré, un HIDS ne voit pas le réseau
- **Elle ne sait pas tout décrire** : des signatures ou des profils à la fois précis et complets sont très difficiles à écrire
- **Elle est spécialisée** : HIDS, NIDS ou sonde applicative (AIDS), chacune ne couvre qu'un type d'attaques

> Plutôt que de chercher la sonde parfaite, on **combine des sondes hétérogènes**

Université de Limoges

Corrélation : le problème

Multiplier les sondes crée un nouveau problème : le **déluge d'alertes**.

- Un simple scan `nmap` peut produire des centaines d'alertes Snort. Une même attaque est vue par plusieurs sondes, chacune avec son propre format. Beaucoup d'alertes sont des faux positifs, et chaque alerte isolée manque de contexte
- Face à des milliers d'alertes par jour, l'opérateur ne peut plus distinguer ce qui est important

> La **corrélation** traite automatiquement ces alertes pour n'en présenter qu'un petit nombre, **pertinentes et compréhensibles**

Université de Limoges

Corrélation : objectifs

Une règle Wazuh relie une alerte Snort à une connexion réussie sur un hôte cela permet donc de :

- **Réduire** le volume d'alertes
  - en regroupant les alertes redondantes ou récurrentes et en éliminant les faux positifs
- **Enrichir** chaque alerte
  - en exploitant la complémentarité des vues (réseau + hôte) et en ajoutant du contexte (vulnérabilités, topologie)
- **Comprendre** l'attaque
  - en reconstituant les scénarios en plusieurs étapes
  - en attribuant des priorités pour offrir à l'opérateur une **vue de haut niveau**

Université de Limoges

Exemple du ver Nimda

```
GET /scripts/root.exe?/c+dir
GET /MSADC/root.exe?/c+dir
GET /c/winnt/system32/cmd.exe?/c+dir
GET /d/winnt/system32/cmd.exe?/c+dir
GET /scripts/..%5c../winnt/system32/cmd.exe?/c+dir
GET /_vti_bin/..%5c../..%5c../..%5c../winnt/system32/cmd.exe?/c+dir
GET /_mem_bin/..%5c../..%c../..%5c../winnt/system32/cmd.exe?/c+dir
GET /msadc/..%5c../..%5c../..%5c/..\xc1\x1c../..\xc1\x1c../..\xc1\x1c../winnt/system32/cmd.exe?/c+dir
GET /scripts/..\xc1\x1c../winnt/system32/cmd.exe?/c+dir
GET /scripts/..\xc0/../winnt/system32/cmd.exe?/c+dir
```

- **Nimda** est un ver exploitant une faille des serveurs IIS
  - Chaque attaque se traduit par une suite de requêtes HTTP
  - Chacune de ces requêtes est anormale/suspecte

Université de Limoges

Détection par Snort

```
root.exe access      64.165.187.170  193.54.194.111  07/20-13:59:32
cmd.exe access       64.165.187.170  193.54.194.111  07/20-13:59:33
cmd.exe access       64.165.187.170  193.54.194.111  07/20-13:59:33
cmd.exe access       64.165.187.170  193.54.194.111  07/20-13:59:33
/_vti_bin/ access    64.165.187.170  193.54.194.111  07/20-13:59:34
cmd.exe access       64.165.187.170  193.54.194.111  07/20-13:59:34
cmd.exe access       64.165.187.170  193.54.194.111  07/20-13:59:35
```

- **Granularité** : les différentes alertes sont indépendantes, bien qu'elles résultent d'une même attaque
- **Sémantique** : aucune de ces alertes n'identifie Nimda
- **Sévérité** : qui est infecté ?
- **Redondance** : toutes les sondes vont avoir toutes ces alertes

Corrélations

- Seulement des alertes ?
  - En fait, non → corrélation d'**événements**
  - Comprend notamment les données contextuelles (topologie, analyse de vulnérabilité, etc.), et les événements produits par d'autres composants (logs de firewall, d'AV, etc.)

> La **corrélation d'alertes** est constituée par l'ensemble des techniques automatisées permettant de traiter et d'analyser les informations de sécurité en vue d'aider les opérateurs de sécurité à prendre les contre-mesures appropriées.

Université de Limoges

Les méta-alertes

- Une **méta-alerte** résulte du regroupement d'une ou plusieurs (méta-)alertes par le processus de corrélation
  - Parfois désignées par hyper-alerts, super-alert ou alerte corrélée
  - Les alertes et les méta-alertes forment une structure hiérarchisée
  - Les feuilles de la hiérarchie sont les alertes « primaires » issues des sondes, les autres nœuds sont des méta-alertes
- **But** d'une méta-alerte
  - Agréger / enrichir le contenu des alertes
  - Présenter une seule alerte à l'opérateur
  - Permettre de suivre les liens entre alertes (analyse post-mortem)

Université de Limoges

Regroupement d'alertes

> Les stratégies de regroupement d'alertes dépendent du processus de corrélation

- Stratégies classiques
  - Regrouper l'alerte avec la base de la hiérarchie
  - Suivre les liens pour trouver la méta-alerte la plus pertinente (parcours en largeur)
  - Problèmes ouverts
    - Considérer tous les regroupements possibles ? Le ou les plus pertinents ?
    - Regrouper les alertes aux "premiers niveaux" ? Aux "derniers niveaux" ?

Université de Limoges

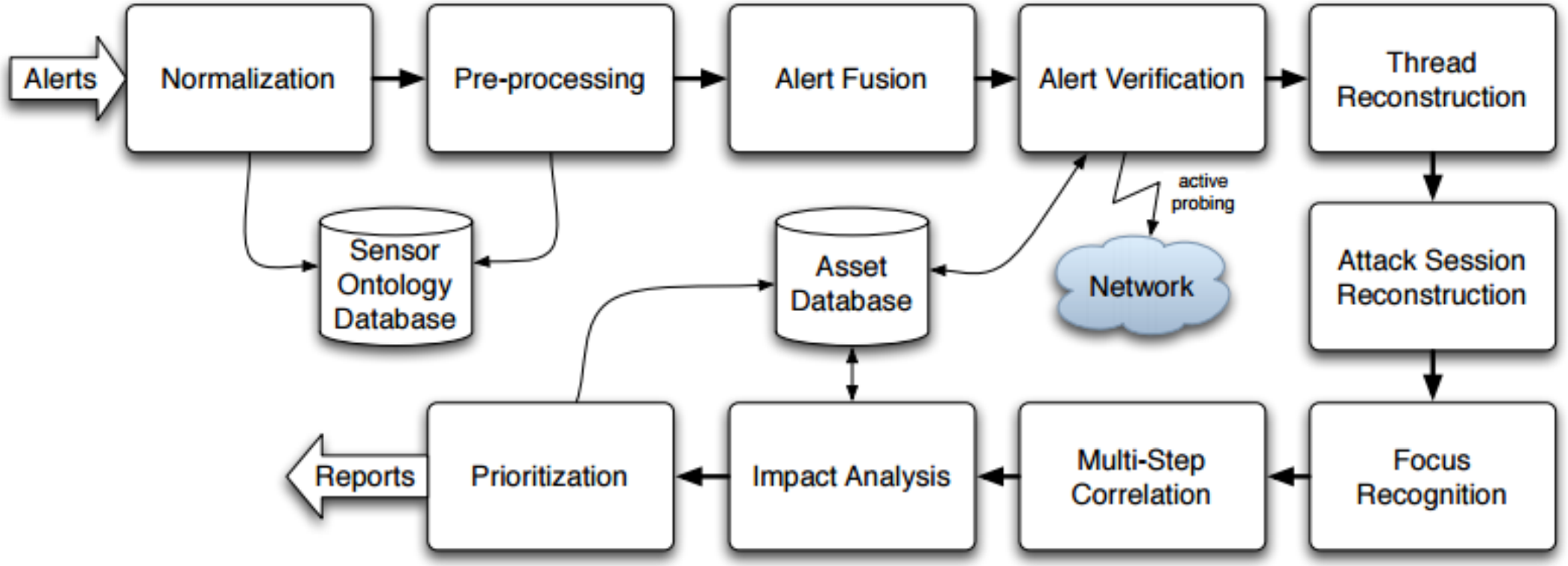
Processus de corrélation

> Pas une seule étape mais une **chaîne de composants**, sans consensus sur sa définition ni sur son implémentation

1. **Mise en forme** : normaliser les alertes dans un format commun
2. **Agrégation** : fusionner les doublons, regrouper par flux ou par session
3. **Vérification** : l'attaque a-t-elle pu réussir sur la cible ?
4. **Scénarios** : reconstituer les attaques en plusieurs étapes
5. **Priorisation** : évaluer l'impact et classer les alertes

Le processus n'est pas strictement séquentiel : parallélisme et rebouclages sont possibles

Processus de corrélation



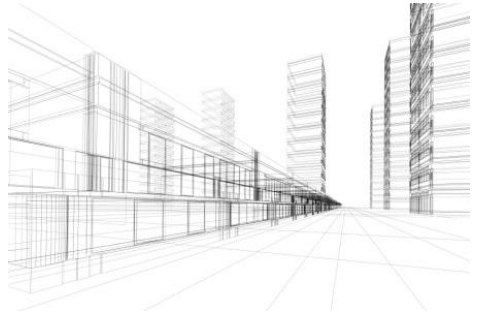
Les 5 étapes détaillées en composants (d'après Valeur *et al.*, IEEE TDSC 2004)

Université de Limoges

Architecture de supervision

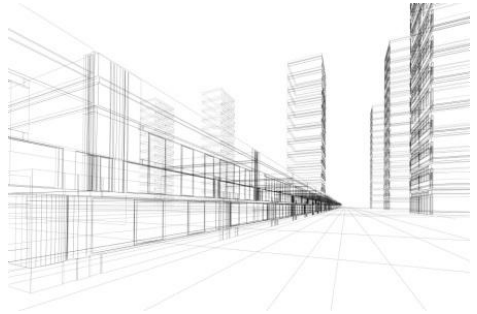
Définition d'une architecture

- D'abord, définir ses besoins (politique de sécurité, analyse de risques / d'impacts)
- **Choix des sondes** :
  - Capacité de détection/prévention
  - Robustesse (attaques contre l'IDS, sûreté de fonctionnement)
  - Performances (paquets/s, connexions simultanées, etc.)
  - Facilité de gestion (documentation, support technique, services, mise à jour, interopérabilité)
  - Coût durant tout le cycle de vie (initial + maintenance)



Définition d'une architecture

- **Choix de l'emplacement** et du type de capture
  - passive ou en coupure
- **Choix de l'interface dédiée pour l'administration**
  - Garantir la furtivité (capture passive)
  - Vulnérabilités de l'interface d'administration



Placement des sondes

- IPS ~ pare-feu : cloisonner des zones de confiance → positionnement aux interfaces, en coupure
- IDS = surveiller une zone
- Définition des zones d'intérêt à surveiller
  - Intérêt : $I = \frac{V}{C}$
  - Valeur : $V = f(\text{impact})$
  - Confiance : $C = f(\text{niveau de protection}, \text{vulnérabilités})$, difficile à estimer
- Plus une zone a de valeur et moins on lui fait confiance, plus il faut la surveiller

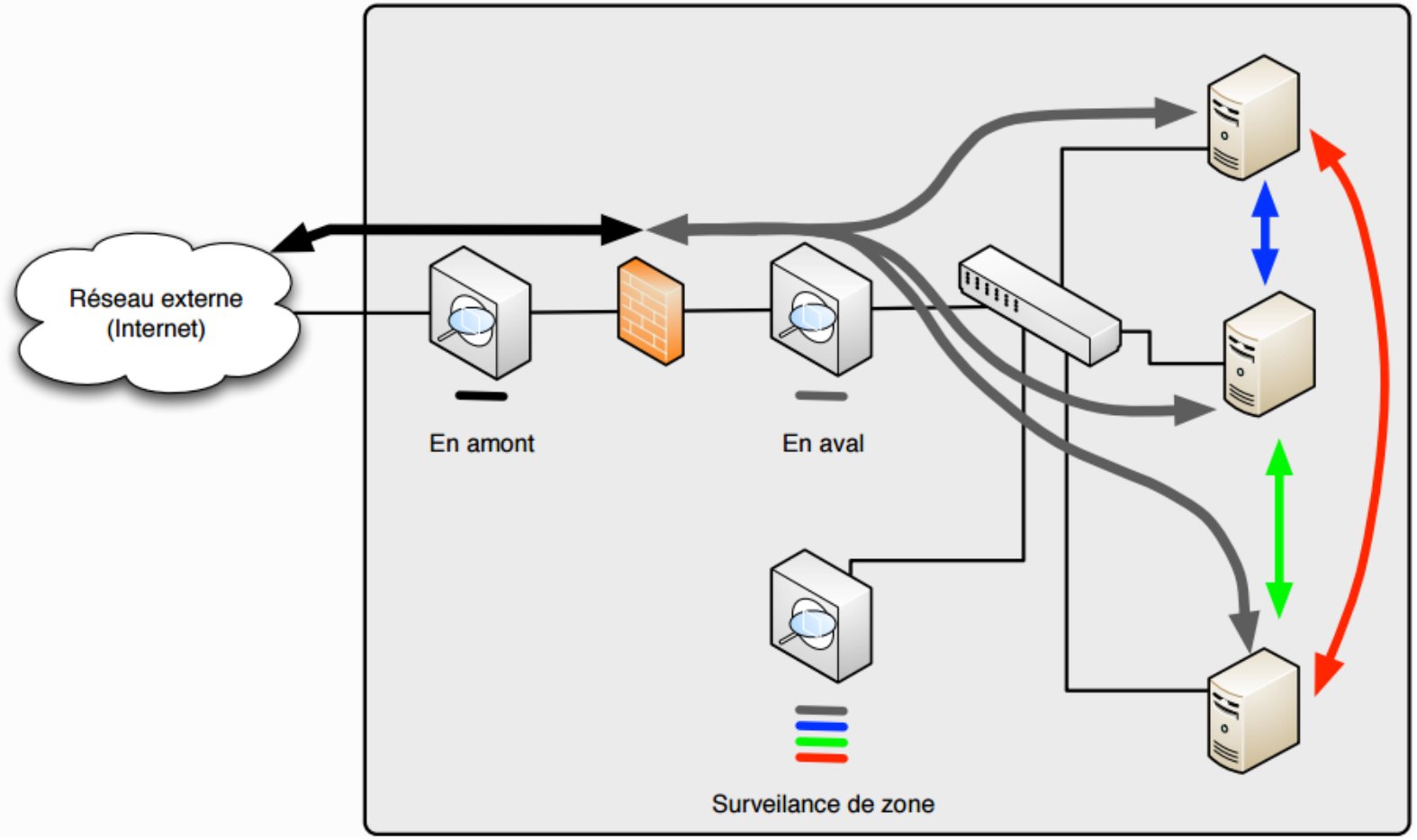


Placement des sondes

- Autre approche (complémentaire) : cloisonner en fonction des applications (pour spécialiser les sondes)
- Placement / pare-feu
  - **En amont** : + visibilité maximale, connaissance de la menace – beaucoup de trafic à analyser, risque d'alertes peu pertinentes, risque d'évasions
  - **En aval** : + moins de trafic, protection, vérification de la politique du FW – certaines attaques sont filtrées
  - **À la place** (mode inline) : revient au cas 1 ou 2 selon configurations



Placement des sondes



Université de Limoges

Bilan sur les outils de détection d'intrusion

Université de Limoges

Bilan : approches des NIDS

Signatures : la plus répandue et la plus mature (Snort, Suricata). Très efficace contre un exploit connu, mais aveugle aux attaques nouvelles et sensible à l'évasion.

Anomalies de protocoles : repérer les écarts à la spécification (RFC) et les valeurs incohérentes. Demande un décodage précis et coûteux.

Anomalies réseau : repérer des flux anormaux (scans, DDoS, propagation de vers). Ne dépend d'aucun exploit, mais l'apprentissage est délicat.

Réputation : listes noires d'adresses IP et de domaines, issues du renseignement sur la menace.

Université de Limoges

Bilan : approches des HIDS

Contrôle d'intégrité : simple et efficace, à condition d'écarter les changements légitimes (mises à jour).

Analyse des journaux et recherche de rootkits : signatures et heuristiques, avec les mêmes limites que côté réseau.

Analyse comportementale des applications et des utilisateurs : détecter les écarts à un profil ou à une politique (SELinux, AppArmor). Puissante, mais source de faux positifs.

> Ces approches sont aujourd'hui réunies dans les **EDR** (*Endpoint Detection and Response*)

Université de Limoges

Panorama actuel

- **NIDS open source** : Snort 3 (Cisco), Suricata (multi-thread), Zeek (analyse de flux)
- **HIDS / XDR open source** : Wazuh, OSSEC
- **EDR commerciaux** : CrowdStrike, Microsoft Defender, SentinelOne…
- **Pare-feu nouvelle génération** (NGFW) intégrant un IPS
- **Solutions spécialisées** :
  - WAF pour les applications Web
  - NDR pour l'analyse comportementale du réseau
  - WIPS pour le WiFi
- **Supervision** : les alertes convergent vers un SIEM, exploité par un SOC

Université de Limoges

Conclusion

Aucune sonde n'est suffisante seule : chaque type de détection a ses **angles morts**.

Une détection efficace repose sur la **complémentarité** des sondes réseau et hôte, placées aux points stratégiques du SI, et sur la **corrélation** de leurs alertes.

> Une alerte n'a de valeur que si elle est **exploitée** : règles maintenues, seuils ajustés et réponse organisée